L2lidar Class – Framework

August 21, 2026

V 2.0.0

(changes from V1.3.5 are highlighted in orange)

(changes from 1.3.6 are highlighted in red)

Author: Mark Stegall

Platform: Qt6.10.2 (Qt Creator), C++

Hardware: Unitree L2 LiDAR (Ethernet / UDP)

L2lidar is the core class that implements a complete UDP communications pipeline, packet reconstruction, decoding, and synchronization logic without any UI dependencies. It is designed to be embedded into GUI or headless applications (Qt timers, ROS bridges, visualization tools, etc.).

It is intended to be used by applications that need to interface to the Unitree L2 4D LiDAR hardware via the UDP Ethernet interface.

Example Apps that use the L2lidar class:

L2diagnostic is a platform-independent C++ framework for validating and diagnosing the operation of the Unitree L2 LiDAR sensor over its Ethernet (UDP) interface. It relies on the L2lidarClass to control and receive data from the L2.

l2lidar_node is a platform-independent C++ framework for a ROS2 publishing node for IMU and point cloud data from the L2.

Goal

The project was developed to replace reliance on undocumented vendor libraries and POSIX-dependent archive binaries provided by Unitree Robotics, enabling transparent packet decoding, timestamp correction, latency measurement, and point-cloud extraction in a portable Qt environment.

Provide a mechanism to override most of the L2 built-in calibration parameters.

Provide a Range correction methodology to adjust for the non-linear range response of the L2.

Define a calibration correction file specification that contains all the information required.

Motivation

Unitree's official SDK distributes:

- Header file
- Example applications
- Precompiled `.a` archive libraries

However:

The archive library source is unavailable:

- It relies on POSIX I/O
- Debugging and porting are challenging
- Protocol behavior is undocumented

This project reconstructs and documents the L2 protocol behavior while providing:

- Robust packet handling
- Error detection and packet loss statistics
- Timestamp synchronization to host time
- Quaternion-based spatial correction
- Extracted point cloud frames for downstream processing
- IMU packets
- Over-ride of built-in factory calibration settings.
- Correction to range measurements based on real world calibration

---

Key Features

UDP Packet Reconstruction

* Combines multiple datagrams into complete L2 packets
* Detects lost and malformed packets

Protocol Decoding

- 3D scan point cloud packets
- 2D scan point cloud packets
- IMU packets
- Version, MAC, IP, work mode, calibration and parameter packets
- ACK packets

Point Cloud Conversion

- ConvertL2data2pointcloud() produces clean `Frame` (vector of `PCpoint`)
  It includes the capabilities to selectively only convert point that are within a specified slow scan angular range. It also includes the ability to convert the range collected into a flattened scan projection in the 3d point cloud space.
- Optional IMU-based quaternion spatial correction for roll & pitch or roll & pitch & yaw
- Includes precomputed range (meters) for each point
- Override of built-in range calibration (allows actual range calibration)
- Accurate timestamps relative to the host
- Accurate conversion of scan data point to cloud point

Starting with V2.x the 2D scan support is frozen. It will not receive new updates or support.

Time Synchronization

- Sync L2 clock to host system time at specified intervals
- Adjustable timestamp scale correction (L2 time is ~1/2 real time)

Latency Measurement (for diagnostics)

- RTT latency using sequence IDs
- EWMA statistics (average, variance, min, max)
- Non-blocking implementation

Command & Control

- Start/stop rotation
- Reset device
- Set work mode
- Configure UDP IP/port
- Set MAC address
- Retrieve firmware version and parameters

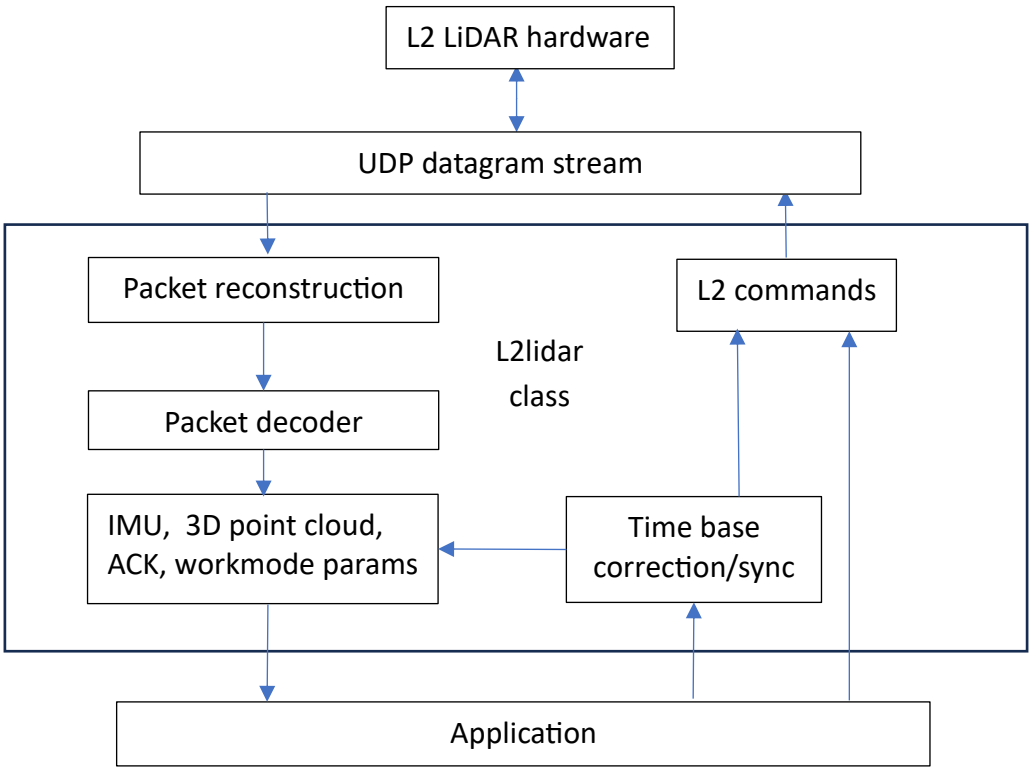
Thread-Safe Data Access

- All decoded packets protected by `QMutex`
- Safe access from GUI or worker threads

Qt-Native Design

- Uses `QUdpSocket`, `QTimer`, `QObject` signals
- No UI code included
- Ready for integration into Qt or ROS2 applications
- This class has only 5 Qt shared library dependencies

Architecture Overview



Dependencies

- Qt (QObject, QUdpSocket, QTimer, QMutex, QByteArray, QVector)
- Qt6 Core shared library (Qt6Core) 6.10.2
- Qt6 Network shared library (Qt6Network) 6.10.2
- Modified Unitree protocol headers:
    - unitree_lidar_protocolL2.h
    - unitree_lidar_utilitiesL2.h
      original source: https://github.com/unitreerobotics/unilidar_sdk2
      BSD-3 Clause

Design Notes

- No UI elements are included in `L2lidar` class
- UART support exists as commented reference code only. UART support has not been implemented
- Communication uses Ethernet (UDP)
- Timestamp correction compensates known firmware clock drift and scale
- Initial IMU and point cloud frames are skipped until time synchronization stabilizes

L2lidar class

Only one instance of this class should be created in application. Ethernet does not allow multiple connections unless multicasting is used. The Unitree L2 is a unicast device not capable of multicasting.

The main class L2lidar is defined in the l2lidar.h include file.
The dependencies are included in the l2lidar.h file as #includes.

L2RangeCorrection.h
L2RangeCorrection.cpp
RangeCalSegments.h
PCpoint.h
quaternion.h

The Unitree external dependencies that are needed by this class:

include “unitree_lidar_protocolsL2.h”
include “unitree_lidar_utilitiesL2.h”

These are modified versions of the original open-source files from the Unitree Lidar SDK2:

unitree_lidar_protocols.h (do not use this version)
unitree_lidar_utilities.h (do not use this version)

The other class dependencies are:

```
#include <QByteArray>
#include <QElapsedTimer>
#include <QMutex>
#include <QObject>
#include <QUdpSocket>
#include <QHostaddress>
#include <QTimer>
#include <QVector>
#include <unordered_map>
#include “quaternion.h”
#include “PCpoint.h”
using Frame = QVector<PCpoint>;
```

L2lidar Class files:

L2lidar.h
L2lidar.cpp

```
// This structure is used for the RTT network latency measurements
typedef struct {
   double lastMeasurement;
   double Average;
   double Variance;
   double min;
   double max;
} Latency;
```

PCpoint.h

```
// This structure is used for point cloud data
struct PCpoint
{
   float x;
   float y;
   float z;
   float intensity; //  0-255
   float range;    // meters (changed from raw L2 range units in meters to post calibrated range
         units in meters)
   float raw_range;  // meters, uncalibrated range reported by L2
   long long time; // timestamp in nanoseconds
   uint32_t ring;
};
```

See **quarternion.h** section for related structures and methods

Class L2lidar

public:

L2lidar(QObject* parent); // constructor
~L2lidar(); // uses default destructor

// Accessors to retrieve the latest L2 packets
const LidarAckData **ack**()
uint32_t **GetL2Workmode**()
const LidarImuDataPacket **imu**()
const LidarIpAddressConfig **IPaddress**() // This has not been observed
const LidarParamDataPacket **L2ParamsPacket**()
const LidarMacAddressConfig **MAC**() () // This has not been observed
const Lidar2DPointDataPacket **Pcl2Dpacket**()
const LidarPointDataPacket **Pcl3Dpacket**()
const LidarTimeStampData **timestamp**() // This has not been observed
const LidarVersionData **version**()

// UDP error reporting
const QString **GetLastUDPError**()

// packet stats from L2 (only updates when L2 socket connected)
// These are not actually critical, only for reporting stats
const Latency **GetLatency**()
void **ClearCounts**(); // clears the packet totals
void **ClearNumPointsConverted**(); // only clear the # of points converted counter

// packets recieved from the L2
uint64_t **lostPackets**()
uint64_t **total2D**()
uint64_t **total3D**()
uint64_t **totalACK**()
uint64_t **totalIMU**()
uint64_t **totalPackets**()
uint64_t **totalOther**()

```
// packets stats retrieved by the app
uint64_t totalIMUretrieved()
uint64_t totalPCretrieved()
uint64_t NumPointsConverted()

// L2 commands
bool GetL2Params(void);
bool GetWorkMode();
bool LidarGetVersion(void);
bool LidarReset(void);
bool LidarStartRotation(void);
bool LidarStopRotation(void);
bool sendLatencyID(uint32_t SeqeunceID);
bool SetL2MAC(LidarMacAddressConfig MACsettings); // requires power cycle
bool setL2UDPconfig(QString hostIP, uint32_t hostPort,
                    QString LidarIP, uint32_t LidarPort,
                    QString GatewayIP, QString SubnetMask); // requires power cycle
bool SetWorkMode(uint32_t mode);

// L2 Timstamp correction and controls
void SetUseSystemNowTimestamps(bool enable);
bool GetUseSystemNowTimestamps();
void EnableL2TimeCorrection(bool enableflag);
bool GetL2TimeScale(long long& ScaleNumerator, long long& ScaleDenominator);
bool SetL2TimeScale(long long ScaleNumerator, long long ScaleDenominator);

// L2 timestamp syncing to host (timer driven)
void EnableL2TSsync(bool enable);
void SetL2TSsyncRate(uint32_t Rate);

// latency measurement
void EnableLatencyMeasure(bool enable);

// Time sync of L2 to host
bool SyncL2Clock() ;    // sync L2 to current system time
 bool SyncL2Clock(TimeStamp timestamp)
```

```
// this is only to set the UDP parameters in the class
// It DOES NOT change the L2 configuration settings
void LidarSetCmdConfig(
        QString srcIP, uint32_t srcPort,
        QString dstIP, uint32_t dstPort);

// UDP socket
bool ConnectL2();  // bind to create, bind socket, connect callback for decode
void DisconnectL2();   // close socket

// Conversion of point frame data from L2 to point cloud
bool ConvertL2data2pointcloud(Frame& frame, bool Frame3D,
                bool IMUadjust,bool AdjustRollPitchOnly,
                double StartScanAngle, double SanAngleWidth, bool flatten,
                bool CalOVR, double CalScale, double CalBias,
                double timeConstraintIMU_PC = 0.07);

  void SetCalibrationOVR(double Scale, double Offset) {
    RangeScaleOVR = Scale;
    RangeBiasOVR = Offset;
  }

  bool GetCalibration(double& Scale, double& Offset){

// overrides
void EnableCalibrationOVR(bool Override);
bool IsCalibrationOVRenabled();

void SetRangeScaleOVR(double Scale) ;
void SetRangeBiasOVR(int32_t Offset) ;
void SetThetaAngleBiasOVR(double angle; // in degrees
void SetAlphaAngleBiasOVR(double angle; // in degrees
void SetBetaAngleOVR(double angle); // in degrees
void SetXiAngleOVR(double angle); // in degrees

double GetRangeScaleOVR();
int32_t GetRangeBiasOVR();
double GetThetaAngleBiasOVR(); // in degrees
```

```
double GetAlphaAngleBiasOVR(); // in degrees
double GetBetaAngleOVR(); // in degrees
double GetXiAngleOVR(); // in degrees
bool ConvertL2data2pointcloud(Frame& frame, bool Frame3D);

// L2 range limits
void SetMinRange_mm(double p) ;
double getMinRange_mm() ;
void SetMaxRange_mm(double p) ;
double getMaxRange_mm() ;
void SetMinTrustedRange_mm() ;
double getMinTrustedRange_mm() ;

// L2 scan paramters
void EnableFlattenScan(bool p);
bool IsFlattenScanEnabled();
double GetStartScanAngle();
void SetStartScanAngle(double p);
double GetScanAngleWidth();
void SetScanAngleWidth(double p);

// L2 non-linear range calibration
void EnableRangeCorrection(bool Correction);
bool LoadRangeCalibration(const std::string& filename);
void ClearRangeCorrection();
bool IsRangeCorrectionLoaded();
bool IsRangeCorrectionEnabled();
const std::vector<std::string> GetRangeCorrectionWarnings();
const std::vector<std::string> GetRangeCorrectionErrors();
const RangeCalibrationInfo& GetRangeCalibrationInfo() const noexcept;

// IMU adjustment to point cloud
bool IsIMUadjustEnabled();
void EnableIMUadjust(bool p);
bool IsAdjustRollPitchOnlyEnabled();
void EnableAdjustRollPitchOnly(bool p);
double GetIMUPCtimeConstraint();
void SetIMUPCtimeConstraint(double p);
```

signals:

void **ackReceived**();
void **imuReceived**();
void **IPreceived**();
void **MACReceived**();
void **L2ParamsReceived**();
void **PCL2DReceived**();
void **PCL3DReceived**();
void **timestampReceived**();
void **versionReceived**();
void **WorkmodeReceived**();

Signals are callbacks used to notify a subscriber that an event has occurred.

L2LDAR CLASS PUBLIC METHODS

Class constructor/destructor

L2lidar::L2lidar(QObject* parent);

~L2lidar::L2lidar();

L2 connect/disconnect methods

These are the equivalent to opening or closing the L2 communications.
To open communications with the L2 you must:

L2lidar::LidarSetCmdConfig()
L2lidar::ConnectL2()

To close you should:

L2lidar::DisconnectL2()

Note: The interface is automatically closed when the L2lidar class is deleted.

bool L2lidar::L2lidar::ConnectL2();

Currently only UDP Ethernet communications is supported.
The UART implementation is only a placeholder for future implementation.
You should set these settings before calling connectL2():

LidarSetCmdConfig() (if not called factory default settings are used)
EnableL2TimeCorrection()
EnableL2TSsync()
SetL2TSsyncRate()
EnableLatencyMeasure()

The first 100 IMU packets and first 100 point cloud packets that are received after the connect are skipped.

NOTE: If you are using WSL2 on Windows 11 the easiest setup of network access is to use mirroring. This the easier rather than using virtual networking. In order to do this, you will need to edit or create the. wslconfig file. It is or should be located in the

directory %UserProfile%. It should look something like:
[wsl2]
networkingMode=Mirrored
firewall=false

If you set firewall=true then you will also likely to set some firewall rules to allow the UDP traffic to/from the L2 for the specified ports.
If you edit this while wsl2 is running you will need to use Power Shell to do a wsl –shutdown and restart wsl. Use can use 'ip a' to verify the network interface.

void L2lidar::DisconnectL2()

This closes the UDP socket. The caller can not receive any data from the L2 until a ConnectL2() is performed again.

UDP error reporting

When the connectL2() or any send command methods fails. They only report true or false. If the return is false this method can be used to retrieve the error message associated with that failure.

const QString **GetLastUDPError**()

Retrieve packet methods

These are used to retrieve the latest packet data. These are used by the caller after receiving a signal notification that a new packet was received for that type of packet. These calls are thread safe.

const LidarAckData **ack();**

Get the last ACK packet received.

uint32_t **GetL2Workmode**();

Get the current L2 workmode

const LidarImuDataPacket **imu();**

Get the last IMU packet received.

const LidarIpAddressConfig **IPaddress**()

Get the latest L2 UDP Address config packet received.
Note: This is implemented but has not been observed.

const LidarParamDataPacket **L2ParamsPacket**()

Get the latest L2 parameters packet.

const Lidar2DPointDataPacket **Pcl2Dpacket();**

Get the last 3D point cloud packet received.

const LidarPointDataPacket **Pcl3Dpacket();**

Get the last 3D point cloud packet received.

const LidarTimeStampData **timestamp()**

Get the latest timestamp received.

const LidarVersionData **version();**

Get the last VERSION packet received.

L2 command methods

These are commands sent to the L2.
They are used to:

change the state of the L2
configure L2
reset the L2
request information from the L2 (such as Version info)

bool **GetL2Params**(void)
This sends a request to the L2 for a Parameters packet. When the L2 sends a parameters packet a signal is emitted, L2lidar:: L2ParamsReceived ();

bool **GetWorkMode**();
This generates a request to the L2 to send a parameter packet to obtain the current workmode.

bool L2lidar::**LidarGetVersion**(void)

This sends a request to the L2 for a version packet.
The are 3 posible results:

1. The command is completely ignored (This can happen early in the startup.)
2. An ACK packet is sent with the status: ACK_WAIT_ERROR. This occurs when the L2 is not fully initializes. It will not be fully initialize until the L2 is no longer in standby and the first point cloud packet is sent. If a standby command is sent after the L2 is full initialized then a version packet will still be sent.
3. A VERSION packet is sent (this class will send a signal to any connected subscribers). An ACK packet will also be sent by the L2 with the status ACK_SUCCESS.

bool L2lidar::**LidarReset**(void)

This causes the L2 to restart (should be equivalent to power cycle).
Some 'workmode' changes are only effective after a restart.
Note:
The L2 restart is immediate and does not send and ACK packet that confirms receipt of the command.

bool **LidarStartRotation**(void)

Sends a run command to the L2.

If the L2 was in standby then it should start scanning it can take >20 seconds to come up to speed and start sending point cloud data.

bool **LidarStopRotation**(void)

Send a standby command to the L2.

This causes the L2 to stop the motors and go into low power mode.

Issues have been seen with not being able to bring the L2 out of standby mode without a power cycle.

bool **sendLatencyID**(uint32_t SeqID);

sets packet sequence ID

This is used in measuring the latency of packets over the UDP interface

It sets a sequence ID that is returned in the ACK packet. This allows an ACK packet to be matched to this specific send command.

Note: Command packets and User command packets use the same packet structure

void **LidarSetCmdConfig**(

QString srcIP, uint32_t srcPort,

QString dstIP, uint32_t dstPort);

This command does not communicate with the L2.

It saves the IP setup required to connect to the L2 through UDP.

This will be extended to include the serial com port if UART communications when is implemented.

This must be set before the L2connect() method is used.

bool **SetL2MAC**(LidarMacAddressConfig MACsettings)

This sets the MAC address of the L2. The L2 factory set MAC address that appears not have a Organization Unique Identifier (OUI) that is assigned. It also does not appear to be serialized from L2 unit to L2 unit. You may want to assign a locally managed MAC address. The minimum requirements are:

MAC[0] bit 0x02 must be set (locally managed MAC address)

MAC[0] bit 0x01 must be cleared (L2 is a unicast device)

MAC[1:5] have not restrictions

bool **setL2UDPconfig**(

QString hostIP, uint32_t hostPort,

QString LidarIP, uint32_t LidarPort,

QStrinf GatewayIP, QString SubnetMask);

This command sets the UDP configuration used by the L2 to send and receive packets through the Ethernet connection to the L2.

Note:

The L2 does not use DHCP. It is really configured for peer-to-peer operation.

This means the host Ethernet connection should also be manually configured.

It does not appear that the L2 uses jumbo packets.

The factory default when it is shipped are:

Lidar IP: 192.168.1.62

Lidar port: 6101 (this is the port L2 listens for packets)

Host IP: 192.168.1.2

Host port: 6201 (this is the port the L2 uses to send packets)

Gateway IP: 0.0.0.0

Subnetmask: 255.255.255.0

If you change these parameters, you will also likely need to change the Ethernet settings on your host to match.

These settings take effect on the next power cycle or reset of the L2.

bool **SetWorkMode**(uint32_t mode)

This commands only sets the workmode in the L2. It does not restart the L2. This must be done separately for certain workmode changes.

Note:

Immediately effective workmode settings that have been observed are:

Std/Wide FOV

IMU disable/enable

Effective after restart or reset

2D/3D mode

serial/UPD mode

start automatically or wait for start command after power on

NOTE: This uses an undocumented command to perform this function

It was discovered using Wireshark to see how the Unitree software sends commands. This is how the Unitree software sets workmode.

The define LIDAR_PARAM_WORK_MODE_TYPE was added to be consistent with the documented commands.

Packet Stats methods

Packet stats from L2 (only updates when L2 socket connected.)
These are not actually critical, only for reporting stats.

void **ClearCounts**()

This resets the packet count statistics.

uint64_t **lostPackets**()

This returns the total number of lost packets received since the last ClearCounts().

uint64_t **total2D**()

This returns the total number of 2D packets received since the last ClearCounts().

uint64_t **total3D**()

This returns the total number of 3D packets received since the last ClearCounts().

uint64_t **totalACK**()

This returns the total number of ACK packets received since the last ClearCounts().

uint64_t **totalIMU**()

This return the total number of IMU packets received since the last ClearCounts().

uint64_t **totalPackets**()

This returns the total number of packets received since the last ClearCounts().

uint64_t **totalOther**()

This returns the total number of other packets received since the last ClearCounts().

uint64_t **totalIMUretrieved**()

This returns the total number of IMU packets retrieved by the app since the last ClearCounts().

uint64_t **totalPCretrieved**()

This returns the total number of point cloud packets retrieved by the app since the last ClearCounts().

uint64_t **NumPointsConverted**()
This returns the total number of point cloud point converted last ClearCounts() or ClearNumPointsConverted().

void **ClearNumPointsConverted**()
This clears just the number of point cloud points converts.

UDP latency measurements

These methods are used to obtain the round trip time latency of UDP packets between the host and the L2.

void **EnableLatencyMeasure**(bool enable)
This enables/disables latency measurements. The disables sending of the L2 latency command. This also means that no ACK packets will be generated.

Latency **GetLatency**()
This gets the most recent UDP RTT latency measurement. The measurements are in msec.

-1.0 is invalid measurement

It return the structure:

```
typedef struct {
   double lastMeasurement; // -1.0 invalid
   double Average; // 0.0 invalid
   double Variance; // 0.0 invalid
   double min; // 999.99 invalid
   double max; // -1.0 invalid
} Latency;
```

The min and max values are the over the entire time period that the L2 is connected using the L2Connect() method.

Time base controls and measurement

The L2 time base does not measure in seconds. It measures in ½ seconds. This is at least true for FW Version 2.8.11.1. The methods are to assist in correcting the L2 time base so time stamps are reported in real world time seconds. These are split into 2 groups.

L2 Time stamp correction and controls

This group is for applying time stamp corrections to incoming IMU, 2D and 3D packets.

void **SetUseSystemNowTimestamps**(bool enable)

This enables the use of the system clock for the timestamp of incoming point cloud and IMU packets instead of using the internal L2 (either corrected or not corrected). This was done to emulate the Unitree L2 SDK software. This is FALSE unless this routine is called with true.

true	IMU and Point cloud packets are timestamped with system now time
false	IMU and Point cloud packets use the L2 timestamp. This can be corrected or uncorrected time based on the time correction settings; **EnableL2TimeCorrection(), SetL2TimeScale(), and EnableL2TSsync()**

bool **GetUseSystemNowTimestamps**()

This returns the current state of the UseSystemNowTimestamps.

return true	IMU and Point cloud packets are timestamped with system now time
return false	IMU and Point cloud packets use the L2 timestamp. This can be corrected or uncorrected time based on the time correction settings.

void **EnableL2TimeCorrection**(bool enableflag)

if enableflag is false then the time base correction is not applied.

if enableflag is true then the time stamp values returned are scaled.

bool **SetL2TimeScale**(long long ScaleNumerator, long long ScaleDenominator)
In hardware that has been checked the Unitree L2's clock timebase runs at ~1/2 real time. This sets the scalar to correct the timebase to match realtime. This allows for fine tuning of the time scaling to compensate for the inaccuracies in the L2 timebase. The default are ScaleNumerator=2, ScaleDenominator=1. This is only applied when **EnableL2TimeCorrection(true)** and **SetUseSystemNowTimestamps(false)**. If either of these conditions is not set to these then the time units are not scaled. There are requirements on the parameter's values also. If they are not met then time units are not scaled.

return true Parameters are valid
return false Parameters are invalid

Parameter requirements:

ScaleNumerator> ScaleDenominator
ScaleNumerator > 0
ScaleDenominator > 0
ScaleNumerator/ ScaleDenominator >= 1.5
ScaleNumerator/ ScaleDenominator <= 3.0

bool **GetL2TimeScale**(long long& ScaleNumerator, long long& ScaleDenominator)
This returns the current time base scale.

return true Parameters are valid
return false Parameters are invalid (see parameter requirements)

L2 Time base syncing to host

This group methods are used to sync the L2 time base to the host time base.
There are 3 likely scenarios here.

a.) Not used, no attempt to sync to host. In this case these methods are not used
b.) The l2lidar runs a timer and regularly syncs the L2 time base to the host time base. In this case only SetL2TSsyncRate() and EnableL2TSsync() are used.
c.) The calling app will manage syncing the host. In this case only the SyncL2Clock(...) methods are used.

void L2lidar::**EnableL2TSsync**(bool enable)
This method enables a sync timer to operate the a specified rate (default is 20HZ). This will sync the L2 time base to the host time base at the specified rate.

bool L2lidar::**SyncL2Clock**()
This syncs L2 to current host system time.

bool L2lidar::**SyncL2Clock**(TimeStamp timestamp)
This set the L2 time base to the value in the timestamp structure.
The TimeStamp structure is (defined in unitree_lidar.protocol.h):

```
typedef struct {
        uint32_t sec;      // time stamp of second
        uint32_t nsec;     // time stamp of nsecond
} TimeStamp;
```

void L2lidar::**SetL2TSsyncRate**(uint32_t Rate)
This method allows changing the sync rate for the time base from the default. The Rate is in milliseconds. The default rate is 50 milliseconds which gives a 20HZ update rate. Higher rates than this starts to affect the UDP packet latencies.

Override of built-in calibration parameters

These methods are used to override the internal calibration values.

The override is enabled/disabled using the method:

EnableCalibrationOVR(true); // use the override values for Range Scale/Bias

EnableCalibrationOVR(false);// use the internal calibration values for Range Scale/Bias

bool IsCalibrationOVRenabled() returns

~~The values for Range Scale and Range bias are set using:~~

~~SetCalibration(Scale,Bias);~~

The following override variables can be set using these methods:

OverideCalibration

minRange

maxRange

RangeBias

RangeScale

AlphaAngleBias

ThetaAngleBias

XiAngle

BetaAngle

For at least one L2 unit FW: 2.8.11.1:

The builtin value Range Scale: 0.000978 // also converts range from mm to meters

The builtin value Range Bias: -365.625 // units: mm

Other units with the same FW have reported different calibration data.

If your point cloud data shows barrel distortion, then make Range Bias more negative. If the point cloud data shows pincushion distortion, then make Range Bias more positive.

~~Readback of override state is done using :~~

~~bool IsEnabled = GetCalibration(&OverrideScale, &OverrideBias);~~

Procedures for adjusting the various overrides is documented in a separate calibration procedure document.

void SetRangeScaleOVR(double Scale)

This sets the conversion scale range values from meters to mm. Unity is 0.001. Typical values are generally less.

double GetRangeScaleOVR()
This returns the current RangeScale override setting.

void SetRangeBiasOVR(int32_t Offset)
This is the range offset in mm. Typical values are -200 to -1000. Using -200 will result in barrel distortion. Using -1000 will result in pincushion distortion. It should be adjusted to remove the barrel/pincushion distortion. RangeScale can then be used to scale for the correct distance.

int32_t GetRangeBiasOVR();
This returns the current RangeBias override setting.

void SetThetaAngleBiasOVR(double angle)
This set the offset for the Theta angle (Azimuth). This can use used to zero the alignment of azimuth. Typical values are in the range 116 to 124 degrees.

double GetThetaAngleBiasOVR()
This returns the current ThetaBias override setting in degrees.

void SetAlphaAngleBiasOVR(double angle)
This set the offset for the Alpha angle (Elevation). This can use used to align the elevation scan. Typical values are in the range +/- 2 degrees. This should be adjusted to align the correct elevation scan angle. This used as the calibration for tilt of the z axis when manufactured.

double GetAlphaAngleBiasOVR()
This returns the current AlphaAngleBias override setting in degrees.

void SetBetaAngleOVR(double angle)
This sets the Beta Angle override setting in degrees.

double GetBetaAngleOVR()
This returns the current Beta Angle override setting in degrees.

void SetXiAngleOVR(double angle); // in degrees
This sets the Xi Angle override setting in degrees.

double GetXiAngleOVR(); // in degrees
This returns the current Xi Angle override setting in degrees.

Signal Methods

These methods are callback functions that are emitted to tell the subscriber that a new packet for that particular type was received. They are notifications only and have no parameters. They use the connect method in the subscriber. The following is an example:

```
connect(&l2lidar, &::L2lidar::PCL3DReceived,
        this, &MainWindow::onNew3DLidarFrame, Qt::QueuedConnection);
```

This example calls the method onNew3DLidarFrame() when the PCL3DRecieved() notification is emitted from the L2lidar class. When the onNew3DLidarFrame() is called it would call Pcl3Dpacket() to obtain the latest 3D packet. Similar connects are used for the other signals. These calls are thread safe.
The IMU, 2D and 3D packets notifications occur at the send rate from the L2 for those packets. The send rate for IMU is approximately ~250 HZ. The send rate for the 3D packets is ~220 HZ. The send rate for the 2D packet is ~50 HZ. The send rate for the time stamp received is approx. ~470 HZ. Time stamps are received in every 3D and IMU packet.
A Version packet are only received when LidarGetVersion() is called.
A ACK packet notification is only received when a packet is sent to the L2 with the exception of a 'reset' command.

Note: TimeStamp packets are sent to the L2 but have not been observed received from the L2.

void imuReceived(); IMU packet received

void PCL3DReceived(); 3D point cloud packet received

void PCL2DReceived(); 2D point cloud packet received

void versionReceived(); VERSION packet received

void timestampReceived(); Time stamp update received

void ackReceived(); ACK packet received

void WorkmodeReceived(); L2 workmode updated

void L2ParamsReceived(); L2 parameter packet received

Data Processing Methods

bool **ConvertL2data2pointcloud**(
Frame& frame,
bool Frame3D,
~~bool IMUadjust,~~
~~bool AdjustRollPitchOnly,~~
~~double StartScanAngle,~~
~~double ScanAngleWidth,~~
~~bool flatten,~~
~~bool CalOVR,~~
~~double CalScale,~~
~~double CalBias,~~
~~double timeConstraintIMU_PC = 0.07~~);

This returns the latest point cloud Frame. It obtains the latest raw point cloud packet sent by the L2 and optionally the latest IMU packet. It then converts the raw data from into a point cloud frame. If the IMU adjust is used the it also applies the pose returned by the IMU to the point cloud frame. The point cloud frame will consist of 'n' points. Each point consists s of x,y,z in meters, the time of the point measurement (in epoch time nanoseconds see Note below), the intensity (0-255) and a ring ID (which is always 1 for the L2). The number of points depends on the scan mode, 2D or 3D, and removal of points that are marked invalid. For 3D data that is up to 300 points per frame and for 2D up to 1800 points per frame. It should be called from the parent class when the parent class received a signal that a new frame is available. See the Signal Methods section.

Note: The timestamp for each point is a long long (64 bits). The units are nanoseconds since the Epoch (Thursday 1 January 1970 00:00:00 UT). This was done so that the timestamp does not suffer precision loss. This allows an app to aggregate frames with precise time references for each point. For further processing in environments like ROS2 the app will likely covert the frame or aggregated frame into a separate capture of the start time (first point timestamp) and then make all the point time stamps relative to the start time. Units do not change, they are always nanoseconds

frame is (QVector<PCpoint>)
Frame3D
true process latest 3D packet
false process latest 2D packet

IMUadjust

true- applies pose from IMU to point cloud data

false do not applies IMU pose correction

AdjustRollPitchOnly

true IMU correction will be roll and pitch

false IMU correction includes roll, pitch and yaw

Note: The IMU correction uses the L2 IMU quaternion. The yaw component is not known to be reliable.

StartScanAngle

The starting slow scan angle in counter clock wise degrees.

ScanAngleWidth

The width of the slow scan in degrees.

Set to 360.0 for all slow scan angles

flatten

false do not flatten

true flatten scans to 2d plane at (StartScanAngle+(ScanAngleWidth/2)

CalOVR

true

Replace the RangeScale and RangeBias calibration values with CalScale and CalBias

false

Use the L2 built-in RangeScale and RangeBias calibration values reported in the point cloud packet.

CalScale

Scalar Range scale correction which Converts raw L2 Range values in mm to floating point in meters.

Typical number is 0.000987

This number adjusts the linear range to to accurately measure distance.

CalBias

Offset correction for the Range scaling in mm.

Typical number is -365.625

Adjusting this number has a direct impact on the barrel/pincushion distort you see in your point cloud. You may want to adjust this is there is obvious distortion in your point cloud. The more negative number moves toward pincushion distortion. The more positive the number moves towards barrel distortion.

~~timeConstraintIMU_PC~~

~~Optional parameter, defaults to 0.07 seconds (70 msec).~~

~~This parameter is only used if IMUadjust parameter is TRUE.~~

~~This is used to decide if the timestamps on the IMU packet and the point cloud packet are close enough to use in the IMU to correct the pose of the point cloud data. This does not change x,y,z translation only the rotation correction to the gravity aligned quaternion in the IMU packet.~~

~~Units are in seconds.~~

return

true	successful
false	if point cloud is empty
false	if point cloud packet does not exit
false	if IMUAdjust is true and no valid IMU data

The are a number of parameters using set/get methods that control the conversion for 3D point clouds. These are:

OverideCalibration
RangeBias
RangeScale
ScanAngleWidth
StartScanAngle
FlattenScan
IMUadjust
AdjustRollPitchOnly
minRange
MaxRange
AlphaAngleBias
ThetaAngleBias
XiAngle
BetaAngle
EnableRangeCorrection

Slow scan angle range

void SetStartScanAngle(double p)
This sets the counter clockwise starting scan angle (theta) for point cloud scan captures. This is in degrees. This only applies when ScanAngleWidth != 360.0

double GetStartScanAngle()
This returns the current value of StartScanAngle in degrees.

void SetScanAngleWidth(double p)
This sets the scan angle width (counter clockwise) in degrees

double GetScanAngleWidth()
This returns the current value for ScanAngleWidth in degress.

A ScanAngleWidth >= 360.0 will results in all scans point being returned. This is the normal settings for collecting complete hemispheres of points.

Specifying a scan width < 360.0 will exclude any points that are outside the scan range. This can also mean that no scan points may be returned for the point cloud. The method will return false if the cloud.size == 0;

Example:
StartScanAngle = 315 degrees
ScangeAngleWidth = 45 degrees
This will return points that are in the scan lines which are +/- 45 degrees from the Y axis. This will not return scans outside that range. This has a subtle effect. For example, a scan at 5 degrees looks the same as a scan at 185 degrees at a macro scale but they are not the same scan in the that the x, y ,z point order is reversed between the 2 scans and the slow scan skew (XY plane) between them is in the opposite direction. This makes it easy to confuse the 2 when you are visualizing them on a display. It also has subtle effects when you are trying to create a line spread range function for calibration.

FlattenScan
Flatten scan collapses all scans to a 2D scan placed at StartScanAngle+ScanAngleWidth/2 in the 3d point cloud space.

void EnableFlattenScan(bool p)
This enables/disables flattened scans.

bool IsFlattenScanEnabled();
This returns current state of the FlattenedScan flag.

This is to assist is the collection of calibration data for examining the range linearity. The idea is to only collect a narrow width of slow scan angles (2-4 degrees) and then flatten them what appears to be a single 2D scan. This also allows for visualization of the range non linearity.

The Frame returned has data converted from the L2 azimuth, elevation, range to x,y,z (in meters). The returned x,y,z coordinate includes the z offset from the mounting surface to the scan origin. This means the returned data is referenced to the center of the L2 mounting surface. Users should note that this is not any of the mounting bolt holes but is the center of the circle of the defining the mounting bolt holes. The mounting bolt holes are also offset by 22.5 degrees from the x,y axis origin.

In V1.35 and earlier the range value returned in the PCpoint structure was the raw uncalibrated range value received from the L2. Starting in V1.3.6 the range value returned in the PCpoint structure is the range value after calibration.

The 'z' offset is defined by the a_axis_dist L2 calibration parameter.

Note: The Unitree software returns a ring ID of 1 for a scan. When the Unitree software aggregates scans the ring ID for each scan is set to the scan aggregation number. Since the scan orientation is a vertical slice not a horizontal slice this is not consistent with the interpretation of ring ID in ROS. Ring ID in this class is always set to 1 so that downstream software knows Rings are not used within the standard ROS interpretation.

IMU adjustment to point cloud

The IMU can be used to apply a quaternion pose adjustment to the point cloud to a gravity based estimated orientation. The quaternion pose is a mathematical construct that represents Yaw, Pitch and Roll attitude of the platform. Yaw is the azimuth rotation, Pitch is the x axis tilt, Roll is the y axis tilt. The L2 has know difficulties with measuring accurate Yaw because of drift. As a result the IMU can also be used to correct only Pitch and Roll and ignore the yaw component. When adjusting the point cloud data it is important to select closely matches (in time) point cloud packets with IMU packets. A time constraint can be specified to the maximum allowable time mismatch. If the time mismatch is exceeded when IMU adjust is enabled then that scan will be dropped.

void EnableIMUadjust(bool p)
This enables/disables the IMU adjustment of the point cloud

bool IsIMUadjustEnabled()
This returns the current state of the IMUAdjust flag.

void EnableAdjustRollPitchOnly(bool p)
This enables/disables the application AdjustRollPitchOnly. This also requires IMUadjust to be enabled.

bool IsAdjustRollPitchOnlyEnabled()
Returns the current state of the AdjustRollPitchOnly flag

double GetIMUPCtimeConstraint()
This set the time constraint for matching the timestamps between the poit cloud packet and the IMU packet.

void SetIMUPCtimeConstraint(double p)
This returns the current value of the time constraint for IMU and point cloud packet matching.

Range Correction and Calibration

The range correction and calibration divides into 3 categories of settings:

- Calibration overrides
- Range correction using a piecewise cubic spline
- Elevation angle LUT

These are read from a Calibration csv file that conforms to the L2 Calibration File Specification V2.0.0.md

The L2lidar class does not generate this file. It only reads it. You can use the app L2diagnostics to generate the calibration file.

void ClearRangeCorrection()
This clears any calibration file range correction and elevation LUT. It also set the LoadCalibrationFile to false and disables range correction.

bool LoadRangeCalibration(const std::string& filename)
This reads the calibration file ‘filename’, validates it, computes a range correction LUT using the spline segments provided in the file, it also set all of the override calibration variables to the values saved in the calibration file.
If this returns false then the GetRangeCorrectionErrors() method returns the specific error message.
GetRangeCorrectionWarnings() can be called to return any warnings generated during the LoadRangeCalibration(). Warning are non fatal which can still be returned even of LoadRangeCalibration() returned true.

const std::vector<std::string> GetRangeCorrectionWarnings();
This returns the last warning messages from the LoadRangeCalibration() method.

const std::vector<std::string> GetRangeCorrectionErrors();
This returns the last error messages from the LoadRangeCalibration() method. This is only meaningful when LoadRangeCorrection() returned false.

void EnableRangeCorrection(bool Correction)
bool IsRangeCorrectionLoaded()
This returns whether a valid RangeCorrection is loaded.

bool IsRangeCorrectionEnabled()
This return the state of the RangeCorrectionEnable flag.

const RangeCalibrationInfo& GetRangeCalibrationInfo()
This returns a structure containing the metadata read during the LoadRangeCorrection() method. It uses the following struct:

```
struct RangeCalibrationInfo
{
   // This structure supports units in either meters or mm
   // but they units can not be mixed
   // They are all meters or all mms
   // Only the constructor of this structure defines which units
   std::string Version;
   std::string Date;

   std::string Sensor;
```

```
    std::string SensorID;
    std::string Firmware;

    std::string CreatedBy;

    std::string RangeCalMethod;
    std::string CalibrationDescription;

    int32_t RangeBias = 0;
    double RangeScale = 0.001;
    double AlphaAngleBias = 1.75;
    double ThetaAngleBias = 120.0;
    double BetaAngle = 0.25;
    double XiAngle = 0.25;

    uint32_t NumberOfSegments = 0;

    double MinRange = 0.0;
    double MaxRange = 0.0;

    double MinTrustedRange = -1.0; // not initialized

    double MinCalRange = 0.0;
    double MaxCalRange = 0.0;

    double RMSResidual = 0.0;
};
```

quaternion.h

This file provides structures and inline methods for quaternion and Euler functions.

structures:

```
struct Quaternion
{
   double w, x, y, z;
};

struct EulerAngles
{
   double roll;
   double pitch;
   double yaw;
};
```

inline methods:

inline void **EulerAnglesRad2Deg**(EulerAngles& e)

Convert Euler angles to degrees from radians

inline void **EulerAnglesDeg2Rad**(EulerAngles& e)

Convert Eluer angles to radians from degrees

inline Quaternion **removeYaw**(const Quaternion& q)

Set the yaw angle component in the quaternion to 0.0

inline void **normalizeQuaternion**(Quaternion& q)

Normalize the quaternion. The quaternion values should follow:

1.0 = sqrt(sum of the squares of w,x,y,z))

inline void **rotateByQuaternion**(const Quaternion& q, float& x, float& y, float& z)

This rotates the x,y,z coordinate in space by using the quaternion. This rotation is equivalent to rotating by roll, pitch and yaw but without the singularity problems in the calculations. The quaternion should be normalized using **normalizeQuaternion**() before calling this. If you want to only rotate by roll and pitch you should call **removeYaw()** before calling this.

inline EulerAngles **QuaternionToEuler**(const Quaternion& q, bool outputDegrees = false)

This converts the quaternion to Euler angles. if outputDegrees is false then the angles are in radians. If outputDegrees is true then the angles are in degress.

Current Revision

V2.0.0 2026-08-21

This is a major upgrade to the capabilities of the L2lidar class.
Calibration overrides: RangeScale, RangeBias, AlphaAngleBias, ThetaAngleBias, Xiangle, Beta angle, minRange, maxRange.
Range correction using piecewise cubic spline.
Formal specification for a calibration correction file.

Revision history

V0.3.7

Preliminary release (accidental title version 0.3.8 Release)

V0.3.8 2026-01-29

Release version
Title correction
Added revision history
Moved requestLatencyMeasurement(); from class public to class private.
Removed signal latencyMeasured(double ms);
Added public method GetLatency();
Added Latency structure
Added in class measurement and calculation of RTT latency.
Added timer to send latency commands a regular interval (4 HZ)

V0.3.9 2026-02-01

Changed latest IMU data to be complete IMU packet
Added capability for L2 Time base correction and controls
Added capability for L2 Time base sync to host clock at regular timer interval
Removed Private class section, will replace once documentation for them is stabilized.

V0.3.10 2026-02-02

Added L2 parameters packet support
Added Get workmode
Added enable/disable latency measurements
minor code cleanup

V0.3.11

Added conversion method to turn L2 point data packets into point cloud frames. (This moves the final processing step into the l2lidar class so it doesn't have to be performed by the parent classes.)

Changed get L2 params to use cmd_value = 3 instead of 1. This matches the observed behavior in Unitree's software.

V0.4.1

Clean up of comments and code
Added Set L2 MAC command.
Added Set L2 UDP config.
Changed Get workmode to use a user command to retrive packet as opposed to the undocumented get L2 params command.
Add a single Get workmode command when the app first connects. This was necessary because of Qt's implementation of the readyRead doesn't necesarily accept UDP packets until at least one UD packet is sent.
Added decoders for the 3 config type packets. This is done even though the UDP and MAC packets have not been observed being sent by the L2.

V0.4.2 2026-02-14

Added error messaging reporting
Bug correction in connectL2(), connect flag was being set to late

V0.4.3 2026-02-18

Added more logic to the timestamping of the point cloud data
mL2EnableSyncHost && enableL2TimeStampFix
    true use L2 timestamping for each cloud point
    false use system time for each cloud point
If adjust cloud packet using IMU is enabled
then if the point cloud packet is different from
IMU packet time by more than 50 msec the point cloud
packet will be dropped.
Added range(m) to point cloud data. It is already present
in the raw point cloud packet. It saves recomputing it later
in a user app. PCpoint.h has been changed to include this field.
Added IMU and PC packet retrieval counts to assist with verifying
packet retrieval by app versus total packets received.
Added more introductory documentation.

V1.0.0 2026-02-20

Separated L2lidar class from the L2diagnostic app and l2lidar_ros2 app
This is the initial release of the standalone L2lidar class

V1.1.0 2026-02-22

Corrected ConvertL2data2pointcloud() to generate more accurate timestamps.
Changed the timestamp units in the cloud point array returned by
ConvertL2data2pointcloud().

PCpoint structure member time change from float to long long

PCpoint structure member time units are now nanoseconds since Epoch

V1.2.0 2026-04-20

Added override of Range Scale and Range calibration params.

V1.3.0 2026-05-12

Changed the timestamp calculations to use long long arithmetic to reduce numerical loss. Allow setting the L2 timebase correction using a/b where a,b are long long types.

V1.3.1 2026-05-17

There are no code changes in this version. This version only affects the documentation. Updates are for:

Networking instructions for wsl2 to use mirroring rather than virtual network.

Clarification of the data returned by the **ConvertL2data2pointcloud()** method.

Minor typo and puncuation.

V1.3.2 2026-05-24

ConvertL2data2pointcloud() now includes optional parameter for time matching constraint between the IMU and point cloud packet when IMUadjust is enabled.

V1.3.3 2026-05-30

Corrected bug in time stamp correction when enable timestamp correction is enabled. This was introduced in V1.3.0 and was inadvertently mixing the IMU and point cloud timestamps. No API changes were made.

V1.3.4 2026-06-15

Added capability to just perform roll and pitch correction using IMU data on a point cloud instead of roll, pitch and yaw.

Added ability to use system timestamp on incoming IMU and point cloud packets.

Corrected initialization conditions when timestamp correction is enabled.

Added quaternion and Euler methods.

Added requirements check on timestamp scaling parameters.

Removed conditional use of system time stamp in ConvertL2data2pointcloud()

V.1.3.5 2026-06-22

Added setting the gateway and subnet mask in the UDP settings for the L2.

V.1.3.6 2026-07-06

The returned value for range in the PCpoint struct was changed from the raw range L2 units to the calibrated range value using RangeBias and RangeScale. The PCpoint

structure also includes a raw_range field for the uncalibrated range returned in the L2 packet.

Corrected typo which stated x,y,z are in mm when they are actually m (meters).

ConvertL2data2pointcloud() added 3 parameters for start scan, scan width and flatten. Changed from calling parseFromPacketToPointCloud() to calling SelectiveParseFromPacketToPointCloud().

Added SelectiveParseFromPacketToPointCloud() to the unitree_lidar_utilities.h file which allows only a specified angular range of slow scan angles to be returned along with a capability to collapse the 3d point cloud to a 2d representation of the scans in a 3d point cloud format using the center of the range of scans for the target 2d representation.

Changed the PointUnitree struct in unitree_lidar_utilities.h to include the field raw_range.